

Underdetermined Systems - Run Instructions

Step-by-step instructions for reproducing the large underdetermined-system experiments from my dissertation

This document explains the practical process I used for the large underdetermined-system benchmark. It is written so that the numerical work can be reproduced even if the person carrying it out is not already familiar with Python, Linux or Slurm. The important point is to follow the stages in order: prepare the files, check the Python environment, submit the numerical calculations on the University of Manchester CSF, check the CSV output, and only then run the plotting program.

The benchmark is a large computational experiment, so the numerical stage should not be treated like an ordinary Python program that is simply opened and run on a laptop. The supplied Python files define the experiment; CSF and Slurm provide the computing resources needed to carry it out.

1. Files required for this section

File	Purpose
Underdetermined.py	The numerical benchmark. It constructs the synthetic underdetermined systems, runs the selected solver and records the measurements.
Underdetermined_Plots.py	The post-processing program. It reads completed numerical CSV files, produces summary CSV files and creates the PNG figures.
Underdetermined_Introduction.pdf	A plain-English explanation of the mathematical problem, experimental studies and solvers.
Underdetermined_Run_Instructions.pdf	This document. It explains how to perform the reproduction from start to finish.
Underdetermined_Requirements_and_Restrictions.pdf	The resource requirements and the settings that should not be changed when reproducing the dissertation experiment.

2. What is being reproduced

The underdetermined benchmark uses synthetic sparse linear systems in which the number of unknowns is larger than the number of equations. In the dissertation, the baseline system has $m = 5000$ rows and $n = 15000$ columns. The experiments then vary one main property at a time: matrix size, sparsity, sparsity pattern, condition parameter, regularisation and spectral distribution. Ten repeated runs are used for each configuration, with a noise level of 0.01 and the random seed set to 42.

The four solver choices in the supplied benchmark are Direct, CG, LSQR and LSMR. Direct was tested as part of the benchmark, but it did not complete successfully for the large underdetermined systems. The reported numerical comparisons for this part of the dissertation therefore use CG, LSQR and LSMR.

The Python program is deliberately separated from the plotting program. The first stage performs the expensive calculation and writes numerical results to CSV files. The second stage uses those CSV files to make the plots and summary tables. This means that the figures can be regenerated without repeating the expensive numerical runs.

3. Access to the CSF

The large benchmark was run on the University of Manchester Computational Shared Facility (CSF). CSF access, authentication, account setup and the University's current login procedure are external prerequisites. I have not included a login tutorial here because those procedures are institutional and can change. Anyone reproducing the work should use the University's current CSF documentation or training to establish a working CSF shell before continuing with these instructions.

Once a CSF shell is available, the rest of the process is ordinary command-line work. The commands below are shown explicitly so that the sequence is clear. Current University CSF rules always take precedence over an example command or Slurm directive in this document.

4. Create a clean working directory

I recommend starting with a clean directory rather than mixing the benchmark with unrelated files. In the CSF shell, create the directory and move into it:

```
mkdir -p ~/underdetermined_systems
cd ~/underdetermined_systems
```

The two Python files should then be placed in this directory. They can be transferred from the repository using the file-transfer method currently permitted by the University. The exact transfer method is not part of the numerical experiment.

5. Check that Python and the required libraries are available

The benchmark uses an Anaconda-based scientific Python environment. The required packages are NumPy, SciPy, pandas and Matplotlib. Before submitting a large Slurm job, I recommend checking that the environment can import all four.

```
python --version
python -c "import numpy, scipy, pandas, matplotlib; print('Scientific Python stack available')"
```

If the second command fails, the Python environment is not ready. The correct response is to use the current University-supported method for selecting or creating a compatible environment. Packages should not be installed blindly into a shared system environment.

6. Understand the experiment names and solver names

The numerical program is controlled by two choices: the experiment to reproduce and the solver to use. The available experiment names are exactly the following:

- Condition Number
- Sparsity
- Matrix Size
- Lambda
- Sparsity Pattern
- Spectrum

The available solver names are:

- Direct
- CG
- LSQR
- LSMR

For example, a single test of the CG solver for the condition-number experiment is called as follows:

```
python Underdetermined.py --experiment "Condition Number" --solver CG
```

The command above illustrates how the Python file is called. It is not intended to replace the complete benchmark array. The full large benchmark consists of all six experiment studies and the four solver choices, giving 24 experiment-solver combinations.

7. Why the Slurm array contains 24 tasks

I organised the large benchmark as a 24-element Slurm array because there are six experimental studies and four solver choices. Each array element represents one experiment-solver combination. This keeps the calculations separate and allows the individual runs to be checked independently.

Array group	Study	Solvers
1-4	Condition Number	Direct, CG, LSQR, LSMR
5-8	Sparsity	Direct, CG, LSQR, LSMR
9-12	Matrix Size	Direct, CG, LSQR, LSMR
13-16	Lambda	Direct, CG, LSQR, LSMR
17-20	Sparsity Pattern	Direct, CG, LSQR, LSMR
21-24	Spectrum	Direct, CG, LSQR, LSMR

A historical Slurm job number is not part of the reproduction. A new job number is normally assigned each time a submission is made, so it should not be copied from an old run or treated as a benchmark parameter.

8. Request suitable Slurm resources

The large underdetermined benchmark is resource-intensive. The allocation used for the dissertation was a 24-hour wall-time request with 128 GB of memory. For the Direct solver, even this amount of memory was not sufficient for the large underdetermined cases to complete successfully. Direct should therefore be regarded as a recorded failed reference attempt rather than a solver that is expected to finish simply by increasing the memory allocation.

For reproduction, the memory and CPU directives must also be checked against the University's current CSF policy. The number of CPUs per task is deliberately not hard-coded in these instructions because the appropriate value is user- and allocation-dependent. Before submitting the job, the researcher should check the current memory limits, CPU limits, partition requirements and any account or project restrictions that apply to their CSF allocation.

A generic resource template is:

```
#!/bin/bash
#SBATCH --job-name=underdetermined_benchmark
#SBATCH --time=24:00:00
#SBATCH --mem=128G
#SBATCH --cpus-per-task=<CPU_COUNT>
#SBATCH --array=1-24

# Activate the University-supported Python/Anaconda environment.

# Map SLURM_ARRAY_TASK_ID to the required experiment and solver.
# The numerical command is then run, for example:
python Underdetermined.py --experiment "Condition Number" --solver CG
```

The CPU placeholder must be replaced with a value permitted by the current CSF allocation. A partition or account directive may also be required. Those details are intentionally left for the person carrying out the reproduction to check against the University's current system.

9. Submit the numerical benchmark

Once the Slurm script has been prepared and checked, save it in the working directory, for example as **run_underdetermined.slurm**. Submit it with:

```
sbatch run_underdetermined.slurm
```

Slurm will return a job identifier. That identifier only belongs to the current execution. It is useful while monitoring the calculation, but it does not need to be stored in the repository or inserted into the Python files.

10. Monitor the array while it runs

The job should be monitored until the numerical tasks have finished. A common Slurm command for viewing jobs is:

```
squeue -u $USER
```

The exact monitoring commands available on CSF can change. The important check is not simply whether the main job has finished, but whether individual array elements completed successfully. A failed array element may leave a checkpoint CSV but no completed `results_final.csv`.

11. Understand the CSV output

The numerical program writes its results beneath an output directory. Each experiment and solver has its own location. During a run, the program can write a checkpoint called **results.csv**. When that particular numerical task finishes, the completed result is written as **results_final.csv**.

```
output_underdetermined/  
  Condition Number/  
    CG/results_final.csv  
    LSQR/results_final.csv  
    LSMR/results_final.csv  
    Direct/results_final.csv  
  Sparsity/  
    ...  
  Matrix Size/  
    ...  
  Lambda/  
    ...  
  Sparsity Pattern/  
    ...  
  Spectrum/  
    ...
```

The exact contents depend on which tasks completed. The final CSV is the file used by the plotting stage. The checkpoint CSV is useful when diagnosing an interrupted or failed calculation because it can show whether any repeated runs were completed before the task stopped.

12. Check the numerical output before plotting

I recommend checking the output tree before running the plotting program. These commands list completed and checkpoint files:

```
find output_underdetermined -type f -name "results_final.csv"  
find output_underdetermined -type f -name "results.csv"
```

If a required **results_final.csv** is missing, do not immediately assume that the plotting script is broken. First check the Slurm output and error files and determine whether the array element stopped because of memory, time, Python environment, numerical convergence or a Slurm configuration issue.

13. Run the plotting program

Only after the completed numerical CSV files have been checked should the post-processing stage be run. Stay in the directory containing the **output_underdetermined** folder and run:

```
python Underdetermined_Plots.py
```

The plotting program searches the underdetermined output tree for completed **results_final.csv** files. It combines the available results, creates summary CSV files and writes the figures to a plots directory. This stage does not repeat the expensive matrix calculations.

14. Find the generated summary CSV files and plots

To see the generated CSV files, use:

```
find output_underdetermined -type f -name "*.csv"
```

To see the generated PNG files, use:

```
find output_underdetermined -type f -name "*.png"
```

The plotting stage produces summaries corresponding to the six studies: condition number, sparsity, matrix size, lambda, sparsity pattern and spectrum. It also creates figures showing quantities such as runtime, iteration count, relative error and the runtime-versus-relative-error relationship. The exact filenames should always be taken from the current version of **Underdetermined_Plots.py** rather than assumed from an old output folder.

15. Copy the results back for inspection

Once the CSF calculations and plotting stage are complete, the generated CSV and PNG files can be transferred to a local computer using the University's permitted file-transfer method. The files can then be opened in a spreadsheet program or viewed as ordinary image files.

The repository itself does not need to contain every generated CSV and PNG. These are reproducible outputs. Keeping them outside the repository makes the repository smaller while still leaving the code and documentation needed to recreate them.

16. What to do if a run fails

A failed task does not automatically mean that the Python code is wrong. Large sparse calculations can fail for several different reasons. The first step is to identify which type of failure occurred.

- **Memory failure:** check the Slurm memory request and the memory available under the current CSF allocation.
- **Time-limit failure:** check whether the task reached the 24-hour wall-time limit.
- **Python/environment failure:** check that NumPy, SciPy, pandas and Matplotlib import correctly in the submitted job.
- **Slurm configuration failure:** check the current partition, account, CPU and array requirements.
- **Numerical failure:** inspect the program output and checkpoint CSV to determine whether the solver stopped during a calculation.

For the Direct solver, failure on the large underdetermined problems is consistent with the dissertation results. The dissertation reports that Direct did not complete successfully even with 128 GB of memory. Increasing memory should therefore not be described as a guaranteed solution for this solver.

17. What must remain unchanged for a true reproduction

The most important distinction in this workflow is between changing the computing environment and changing the experiment. CSF policies may require a different partition, CPU count, account directive or other resource setting. Those are execution-environment changes. The mathematical benchmark should not be altered merely to make the job easier to run.

- Keep the matrix dimensions defined by the supplied implementation.
- Keep the parameter ranges for all six experimental studies.
- Keep the sparse matrix construction and the specified sparsity patterns.
- Keep the condition-parameter, regularisation and spectral settings.
- Keep the random seed and repeated-run structure.
- Keep the solver formulations and stopping settings implemented in the supplied Python code.
- Do not silently replace the large sparse matrices with dense arrays.
- Do not change a solver, parameter range or matrix construction method and then describe the resulting run as an exact reproduction.

If an execution setting has to be changed because of a current CSF restriction, that change should be recorded when reporting the reproduction. This makes it possible to distinguish a difference caused by the computing environment from a difference caused by changing the experiment itself.

18. Recording the computing environment

For a useful reproduction record, I recommend noting the Python version, the main scientific-library versions, the CSF partition used, the requested memory, the CPU count per task and the wall-time limit. These details do not change the mathematical definition of the experiment, but they help explain differences in runtime between executions.

Exact runtimes should not be expected to match perfectly between two executions. Processor type, BLAS and LAPACK implementations, SciPy versions, system load and other hardware or software differences can affect timings. Small floating-point differences are also possible. The more important requirement is that the benchmark definition and solver implementation remain the same.

19. Final reproduction checklist

- CSF access has been established using the University's current procedure.

- Underdetermined.py and Underdetermined_Plots.py are present in the working directory.
- The required Python scientific libraries import successfully.
- The current CSF memory, CPU, partition and account requirements have been checked.
- The numerical stage has been submitted as the intended 24-task array.
- The array has been monitored and failed elements have been identified.
- Completed results_final.csv files have been checked.
- Underdetermined_Plots.py has been run only after the numerical results were available.
- The summary CSV files and PNG figures have been checked.
- Any required generated outputs have been copied back for inspection.
- Any changes to the execution environment have been recorded.
- No benchmark parameter has been changed without being explicitly documented.

20. Repository contents after reproduction

The purpose of this repository folder is to provide the material needed to reproduce the work, not to store every output generated during a run. After the results have been checked and archived separately if required, the generated CSV and PNG files may be removed. The reproducibility package should retain the two Python source files and the three explanatory PDFs.

The intended workflow is therefore: read the introduction, check the requirements and restrictions, follow these run instructions, generate the numerical CSV files on CSF, run the plotting program, and then inspect the recreated figures and summary tables. This is the sequence I used to separate the computational experiment from the presentation of its results.